

AI Emergency Operations & Resource Orchestration System

An evidence-aware operational decision engine that turns fragmented, conflicting disaster reports into explainable, continuously replanned response.

Solution Proposal & Implementation Status Report

- Evidence fusion with uncertainty
- Conflict detection
- Graph accessibility routing
- Constrained resource optimisation
- 14 IBM Bob MCP tools
- Deterministic & auditable

PROJECT

Bhote Valley Emergency
Simulation
bob-ai-hackathon-shutdown

TEAM

Om Prajapati (Lead)
Jeet Pandya · Prince Patel
Darshan Mandalaywala

DOCUMENT STATUS

Verified against source tree
18 September 2026

— Contents

1	Executive Summary	What was built, what is verified, what remains
2	The Problem	Information fragmentation in disaster response, and why current tooling fails
3	Proposed Solution	Design principle, the seven-stage decision pipeline, the AI/deterministic boundary
4	System Architecture	Five layers, component responsibilities, data flow
5	Implementation Status	Module-by-module inventory of everything already working
6	Algorithm Reference	Exact formulas, weights and constants as coded
7	Scenario & Benchmark Harness	The 13-event Bhote Valley simulation and AI-vs-baseline comparison
8	Verification Status	Test coverage, what is proven and what is unproven
9	Gaps, Drift & Limitations	Honest accounting of documented-but-absent features and known risks
10	Roadmap	Prioritised next steps
11	Appendix	Repository layout, how to run, dependency inventory

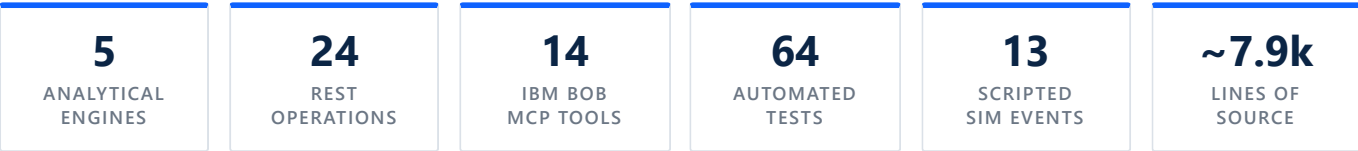
ABOUT THIS DOCUMENT

This report has two jobs. First, it states the proposed solution — the problem being solved, the design principle behind it, and the architecture that follows from that principle. Second, it gives an accurate, source-verified account of what is already implemented, so that the proposal can be read against reality rather than intent.

Every count, formula, weight and threshold quoted here was read directly from the code in this repository, not from its documentation. Where the existing documentation and the code disagree, the code is treated as authoritative and the discrepancy is recorded in Section 9. No code was modified in the preparation of this document.

1 Executive Summary

We propose an evidence-aware operational decision engine for disaster response: a system that accepts conflicting, ageing, partially-reliable field reports as its normal input, quantifies what is and is not known, and produces continuously replanned resource allocations that a human coordinator can interrogate and override. The system is built and running end to end.



The core idea

Conventional emergency tooling assumes information is reliable and treats disagreement as an error state. In real disaster response, disagreement is the normal state: a field officer reports a bridge passable at 06:00, a community radio operator reports it swept away at 08:30, and a satellite pass from 04:45 shows rising water. Our design principle is that **uncertainty is a first-class data type, not an afterthought**. Every observation is retained, weighted by source reliability, self-reported confidence and exponential freshness decay, and fused into a single estimate that carries its own confidence and an explicit conflict flag. Nothing downstream — priority scoring, route planning, resource assignment — is allowed to forget how well-supported its inputs were.

What is implemented

All five analytical engines are complete and deterministic: evidence fusion with conflict detection, criticality and priority scoring, a NetworkX accessibility graph with bridge-capacity and blocked-edge handling, a greedy-plus-local-improvement resource optimiser with a nearest-resource baseline for comparison, and an event-driven simulation engine that re-runs the entire pipeline after every event. These sit behind a FastAPI service exposing 24 operations, a React 19 operational dashboard with a live Leaflet map, and a 14-tool MCP server that lets IBM Bob answer operational questions from real backend state rather than canned text.

Component	Status	Evidence
Evidence fusion engine	COMPLETE	202 lines; weighted fusion, value canonicalisation, 0.3 conflict threshold, verification recommendation
Priority & criticality engine	COMPLETE	244 lines; both weight sets sum to 1.0, asserted by test
Graph accessibility engine	COMPLETE	275 lines; NetworkX DiGraph, Dijkstra on a filtered subgraph, bridge dominance over road state
Resource optimiser + baseline	COMPLETE	360 lines; capability matching, infeasible-route exclusion, pairwise swap improvement
Simulation engine	COMPLETE	855 lines; 13-event script, custom injection, full reset, 10 event-type state handlers
FastAPI service	COMPLETE	916 lines; 24 operations across 23 paths
IBM Bob MCP server	COMPLETE	802 lines; 14 tools, JSON-RPC 2.0 over stdio, all proxying live HTTP
React dashboard	COMPLETE	~1,900 lines; 11 components, Leaflet map, 5-second polling, event injection UI
Benchmark harness	COMPLETE	208 lines; 18 metrics, AI plan vs nearest-resource baseline, no hardcoded results
Operator confirm / reject write-back	NOT BUILT	No <code>POST /actions/allocate</code> ; plans are advisory-only today
Durable audit trail	PARTIAL	In-memory audit log works and is served; the <code>audit_log</code> table is never written
Live push to dashboard	NOT BUILT	No SSE/WebSocket; the dashboard polls every 5 s instead
Demo video & hosted deployment	NOT DONE	<code>demo-video-link.txt</code> and <code>live-demo-url.txt</code> are both placeholders

THE DIFFERENTIATOR

When bridge B1 receives two contradictory field reports fifteen minutes apart — *partially blocked* at confidence 0.82, then *open* at confidence 0.50 — the system does not silently pick a winner. It computes weighted support for each competing value, finds the runner-up holds at least 30 % of total evidence weight, flags the subject as `conflicting` , lowers the confidence propagated to every downstream consumer, and emits a recommendation to verify in the field before dispatching. That conflict then depresses B1's contribution to priority scoring and inflates the effective routing cost of the road it carries. The uncertainty travels with the decision.

WHAT THIS IS NOT

This is a decision-support layer, not an autonomous dispatcher. The optimiser emits recommendations with rationale; `DecisionExplanation.human_verification_required` defaults to `True` in the domain model and is never overridden anywhere in the codebase. It is also a closed simulation: there is no live sensor, satellite or telephony ingestion, and every entity, population figure and event in the scenario is synthetic.

2 The Problem

2.1 Information fragmentation is the first casualty

When a major disaster strikes a remote mountainous region, reliable information fails before anything else does. Floods, landslides and earthquakes simultaneously sever roads, cut power, disable cellular infrastructure and displace the very personnel responsible for reporting conditions. What arrives at an emergency operations centre is not a picture of the situation; it is a stream of fragments that disagree with each other and with themselves over time. No single report is complete, no single source is authoritative, and the synthesis has to happen in minutes.

The coordinator's actual task is therefore not data retrieval. It is inference under time pressure across three simultaneous unknowns: **what is true** (which of these conflicting reports to believe), **what is reachable** (whether the route implied by a dispatch decision still exists), and **what is worth doing next** (which of seven deteriorating sites earns the one remaining ambulance). Each of those three is tractable in isolation. Together, under fatigue, they are where allocation decisions go wrong.

2.2 The motivating scenario

Our reference case is modelled on the geography and logistics of a Nepali river valley: a densely settled corridor flanked by steep ridgelines, served by a small number of routes whose bridges are single points of failure. A monsoon flood triggers overlapping crises across five villages, a district hospital, two health posts, two schools, a shelter and a hydropower station within a three-hour window. Eight response units — three rescue teams, two ambulances, an engineering team, a supply vehicle and a medical unit — must be allocated while bridges fail, roads block, a rescue team goes mechanically unavailable, and field reports contradict one another.

SYNTHETIC DATA DISCLAIMER

All entities, coordinates, population figures, events and benchmark values in this project are entirely synthetic and generated for simulation purposes. They are proportioned to reflect realistic operational complexity but do not reconstruct any actual event, location or casualty data.

2.3 Why existing tooling falls short

Approach	What it does well	Where it breaks
GIS dashboards	Visualise incident pins, asset locations and resource positions on a map	They display, they do not reason. A dashboard can show a bridge status of "unknown"; it cannot weigh the age of that report against the reliability of its source and the urgency of the destination. All synthesis is externalised to an operator who is already saturated.
Conversational assistants	Answer questions and retrieve records in natural language	A chatbot waits to be asked. It cannot monitor a continuously updating state, notice that an approved route has become impassable, or recompute allocations when a new report lands. Disaster response cannot wait to be asked.
Commercial routing engines	Compute fast, accurate shortest paths under normal conditions	They optimise distance and time, not accessibility under degradation. They do not model a road rated passable at hour one and impassable by hour three, and they have no representation for a route that is technically open but operationally unsafe because its only bridge has a structural confidence of 0.4.
Unaided human prioritisation	Contextual judgement, local knowledge, accountability — irreplaceable	Across seven or more simultaneous incidents with degrading information quality, cognitive load, anchoring bias and fatigue produce allocations that systematic analysis measurably improves. The goal is to support this judgement, not replace it.

2.4 The gap we address

The gap is not a shortage of data or of technology. It is the absence of a layer that can do five things at once:

1. **Fuse conflicting, uncertain, time-decaying observations** into a coherent picture that carries its own confidence
2. **Model infrastructure accessibility dynamically**, as conditions and the evidence about them evolve
3. **Optimise constrained allocation** across competing priorities and incompatible resource types
4. **Replan continuously** as each new observation arrives, rather than producing a single plan
5. **Explain itself in natural language**, with every recommendation traceable to the observations that produced it

3 Proposed Solution

An evidence-aware operational decision engine. It sits between raw observation streams and the coordinator, and its only job is to convert fragments into defensible, confidence-weighted, continuously updated guidance.

3.1 Design principle

The system does not wait for a complete and consistent picture before producing recommendations. It treats every observation as partial evidence, quantifies what is known and unknown, and produces the best available guidance given the current information state — while making its confidence and its reasoning legible to the human who must act. This distinguishes it from passive dashboards, which present data without reasoning, and from autonomous agents, which act without adequate oversight.

Two consequences follow directly from that principle, and both are implemented. First, **confidence and accessibility are priority inputs, not annotations**: a critically affected village reachable only by a low-confidence route scores differently from an identical village with a confirmed clear route. Second, **the analytical core is deterministic**: evidence weighting, scoring, routing and assignment are closed-form or algorithmic, so every number is reproducible and auditable. The language model is confined to the interface.

3.2 The decision pipeline

Seven stages run in sequence. Stages 2 through 6 re-execute in full after every single event, which is what makes the replanning continuous rather than periodic.

- 1 Observation intake**

Reports arrive from nine source classes — satellite, drone, field report, emergency call, hospital, school, sensor, authority report, simulation — each carrying a timestamp, a self-reported confidence, a subject entity and an observation type. Nothing is discarded and nothing is trusted unconditionally. Observations are persisted with full provenance.
- 2 Evidence fusion with uncertainty**

Observations sharing a subject and attribute are fused into one estimate. Each contributes $\text{source_reliability} \times \text{confidence} \times \text{freshness}$, where freshness decays exponentially with age at a rate tuned per observation type. Competing values are canonicalised into comparable categories, so *impassable*, *collapsed* and *closed* all count as support for *blocked*. The winner is the value with the greatest weighted support, and it carries a derived confidence rather than inheriting one.
- 3 Conflict detection**

If the runner-up value holds at least 30 % of total evidence weight, the subject is flagged **conflicting**, the specific dissenting observation IDs are attached, and a field-verification recommendation is generated. The flag then reduces the confidence factor feeding priority scoring, so an unresolved conflict measurably changes the plan instead of merely appearing in a log.
- 4 Situation assessment and scoring**

Fused evidence drives two distinct scores per site. **Criticality** captures intrinsic severity from population, service class, time sensitivity, isolation and damage. **Priority** then folds in what can actually be done, adding urgency, impact, accessibility and evidence confidence. Keeping them separate is deliberate: a site can be critically affected yet correctly deprioritised this hour because no route to it clears the confidence bar.
- 5 Dynamic accessibility graph**

The road network is a directed graph whose edge costs encode current condition, not just distance. Blocked edges are removed from consideration entirely; unknown, high-risk and partially-blocked edges carry multiplicative penalties. Bridge state dominates the road that carries it — a blocked bridge blocks its road outright — and vehicle weight class is enforced against bridge capacity, so a heavy ambulance is refused a medium-rated crossing that a rescue team may use.
- 6 Constrained resource optimisation**

Tasks are generated from the scored situation, then matched to resources under three hard constraints: capability compatibility, resource availability, and route feasibility. Pairs whose route does not exist are eliminated before scoring, which is how blocked-route violations are structurally prevented rather than merely penalised. A greedy pass produces an initial one-to-one assignment; pairwise swaps then refine it where they beat the incumbent by more than 5 % on priority-weighted travel time.
- 7 Explanation and human confirmation**

Every assignment carries the priority score that motivated it, the constraints that shaped it, the alternatives that were rejected, and a confidence value. IBM Bob renders these as prose on request. The recommendation is advisory by design; the human decides.

3.3 The AI / deterministic boundary

This boundary is the most important architectural commitment in the system, because it is what makes the output defensible after the fact.

Responsibility	Implementation	Why
Intent parsing, prose generation	LLM (IBM Bob / watsonx)	Natural language is the model's native domain and carries no analytical risk
Evidence weighting & fusion	Deterministic formula	Must be auditable; a coordinator can recompute any weight by hand
Conflict detection	Deterministic threshold	Consistent across runs; zero hallucination surface
Criticality & priority scoring	Deterministic weighted sum	Reproducible, and the weights are domain-tunable by responders rather than opaque
Route planning	Dijkstra on a filtered graph	Correct by construction and fast
Resource assignment	Greedy + local improvement	Explainable per assignment, with a measurable baseline to compare against

STATED AS A RULE

Evidence quality assessment and scoring are never delegated to a language model. Bob's role is interface translation in both directions: turning a coordinator's question into a structured tool call, and turning a structured response into readable prose. If watsonx credentials are absent or the call fails, a deterministic provider generates the same summaries from templates and the analytical output is unchanged.

4 System Architecture

Five layers. Each has one responsibility and a defined interface with its neighbours. The engine layer is deliberately free of I/O concerns, which is what makes all five engines unit-testable in isolation — and they are.



4.1 Replanning cycle

The single most important control-flow fact about the system: after every event, whether scripted or operator-injected, the backend runs the same four-stage pipeline before returning. There is no partial or incremental update path, which trades some efficiency for the guarantee that state is never internally inconsistent.

```
# app/engines/simulation.py – executed after every processed or injected event
_process_event(evt)      → create observation, mutate asset / road / bridge / resource state
_fuse_all()              → re-fuse every (entity, observation_type) group
_recalculate_priorities()→ rescore all 13 non-infrastructure assets
_generate_tasks()        → discard and rebuild the task set from current state
_run_optimizer()         → new AI plan + baseline plan, persist decisions
```

4.2 State ownership

A module-level `WorldState` object is the authoritative in-memory store: assets, roads, bridges, resources, observations, the fused-evidence cache keyed by `entity:observation_type`, priorities, tasks, the current and baseline plans, simulated clock position, and the audit log. SQLite is the durable record for the entities, raw observations, decisions and the event script. Derived artefacts — fused evidence, priorities, tasks and plans — are recomputed rather than stored, which means a restart replays cleanly from the observation record.

ARCHITECTURAL NOTE: THE CLOCK

The existing `docs/architecture.md` describes a tick loop advancing simulated time in configurable 15-minute intervals. The implementation is **event-driven, not ticked**: processing an event jumps the simulated clock directly to that event's `time_offset_min`. The 15-minute spacing is a property of the scenario file, not of a configured tick. The behaviour is equivalent for the scripted scenario and simpler to reason about, but the document should be corrected.

5 Implementation Status

Everything in this section was read from source in this repository. Line counts are exact; capability claims are traced to the file that implements them.

5.1 Code inventory

File	Lines	Responsibility
backend/app/main.py	916	FastAPI app, 24 route operations, request models, lifespan seeding, static frontend mount
backend/app/engines/simulation.py	855	<code>WorldState</code> , event dispatch, 10 state-change handlers, task generation, reset
backend/app/engines/resource_optimizer.py	360	Capability map, pair scoring, greedy assignment, pairwise swap, nearest-resource baseline
backend/app/models/domain.py	381	10 enums and 16 Pydantic v2 models — the shared vocabulary
backend/app/utils/ai_provider.py	315	Provider abstraction, watsonx client, deterministic fallback, keyword Q&A router
backend/app/engines/graph_engine.py	275	NetworkX DiGraph, edge cost model, bridge dominance, vehicle fit, routing
backend/app/engines/priority_engine.py	244	Criticality, priority, urgency / isolation / damage / confidence derivation, explanations
backend/app/engines/benchmark.py	208	18-metric benchmark, AI vs baseline, improvement computation, methodology block
backend/app/engines/evidence_fusion.py	202	Freshness, weighting, canonicalisation, fusion, conflict detection
backend/app/db/database.py	183	Async engine, session factory, 8 ORM table definitions, <code>init_db</code>
backend/app/db/seed.py	143	Scenario loading with path fallbacks, idempotent seeding
backend/app/config.py	98	Every tunable parameter in one place — reliabilities, decay rates, weights, thresholds
backend/tests/	~1,000	43 unit tests + 21 integration tests
mcp/server.py	802	14-tool registry, JSON-RPC 2.0 stdio loop, HTTP proxy to the backend
frontend/src/App.tsx	896	All 11 React components, polling loop, layout, Leaflet map
frontend/src/{api.ts, main.tsx, *.css}	1,021	Typed fetch client with <code>ApiError</code> , entry point, hand-written CSS design system

Approximately 7,900 lines of application source excluding lock files, generated assets and documentation.

5.2 Evidence fusion engine — complete

Implements the full weighting chain. Freshness is `exp(-λ × age_hours)` with age measured against the simulated clock and floored at zero. Weight is the product of source reliability, the observation's own confidence, and freshness. Before competing values are compared they are canonicalised into ten categories, so semantically identical reports phrased differently accumulate support together instead of splitting it.

Fused confidence is the product of two terms: the fraction of total weight supporting the winning value, and the mean weight of the observations that support it. This is why a lone stale low-confidence report yields low confidence even

though it is unopposed. Confidence is then banded into high / medium / low / very low for display. An empty observation set returns `unknown` at confidence 0.0 rather than raising.

PROPERTY WORTH KNOWING

Because the second term is a mean rather than a sum, corroborating duplicates do not raise confidence — three identical observations of weight w still yield roughly w . Additional independent sources raise confidence only by shrinking the competing fraction. This is conservative and safe, but the value should be read as a support score rather than a calibrated probability.

5.3 Graph accessibility engine — complete

A NetworkX `DiGraph` in which nodes are asset IDs and each road becomes two directed edges, so the 14 seeded roads yield 28 traversable edges. Routing builds a filtered edge subgraph that excludes three classes of edge — vehicle-restricted, bridge-capacity-insufficient, and blocked — then runs Dijkstra weighted by effective cost. Every excluded road is returned to the caller in `blocked_alternatives`, so the answer explains what was ruled out and not merely what was chosen. Infeasibility is a first-class return value carrying a reason string, never an exception.

Bridge state dominates the road that carries it. A blocked bridge forces its road to blocked at risk 1.0; a high-risk, unknown or partially-blocked bridge raises the road's risk to the corresponding penalty; and road confidence becomes the minimum of road and bridge confidence. Vehicle fit is enforced on an ordered capacity scale, so a *heavy* unit is refused a *medium* bridge while a *medium* unit is permitted.

5.4 Priority engine — complete

Both scores are implemented with weights read from configuration, and both weight sets sum to exactly 1.0 — an invariant asserted by a dedicated unit test rather than assumed. Static per-entity-type tables supply service criticality and time sensitivity for twelve entity types, with a 0.5 default for anything unrecognised. Population is normalised against a 10,000-person reference.

The derived inputs are all implemented and all evidence-driven: **urgency** takes the maximum over a mapping of evidence values (destroyed 1.0 down to damaged 0.60) and asset status; **isolation risk** reads confirmed isolation directly and treats a high-confidence blockage as 0.7; **damage severity** multiplies a per-value severity by that value's confidence and takes the maximum; and the **confidence factor** is mean evidence confidence reduced by 15 % per active conflict, defaulting to 0.3 where no evidence exists at all. Every score ships with a generated explanation string and a list of supporting factors, including an explicit conflict count.

DIRECTIONAL DESIGN CHOICE

Accessibility and confidence are *positive* priority terms. An unreachable site therefore loses up to 0.15 of its score and a poorly-evidenced site loses up to 0.10. This encodes “prioritise what we can actually act on, with evidence we can actually trust” — and it is why criticality is kept separate, so the underlying need of a deprioritised site remains visible rather than being erased.

5.5 Resource optimiser — complete

Greedy assignment with pairwise local improvement. Eligibility requires both that the resource is available and that the task type appears in that resource type's capability list; the five-entry capability map is explicit, so an ambulance cannot be sent to an engineering task. Routing is performed per candidate pair using the vehicle class implied by the resource type, and pairs whose route is infeasible are dropped before scoring — structural prevention of blocked-route dispatch rather than a soft penalty.

Pairs are scored as $\text{priority} \times (0.4 + 0.6 \times \text{time_efficiency})$ where time efficiency decays linearly to zero at a 120-minute travel ceiling. The greedy pass takes pairs in descending score order with strict one-to-one matching. Local improvement then runs up to ten passes of pairwise swaps, accepting a swap only when it beats the incumbent by more than 5 % on priority-weighted travel time. Each assignment produces a `DecisionExplanation` carrying reasons, constraints, rejected alternatives and a confidence value.

A separate `baseline_nearest_assignment` implements the naive comparator: take tasks in priority order, give each the feasible resource with the shortest travel time, no refinement. This is what the benchmark measures against, and it is a genuine algorithm rather than a synthetic penalty applied to our own result.

5.6 Simulation engine — complete

Event-driven rather than ticked. Processing an event advances the simulated clock to that event's offset, creates the corresponding observation with full provenance, persists it, dispatches to a state-change handler, then runs the four-stage replanning pipeline. Ten event types have dedicated handlers covering bridge damage, conflicting bridge reports, confirmed bridge blockage, road blockage, alternate route opening, resource unavailability, hospital demand surge, school evacuation, satellite observation and confirmed village isolation.

Operator-injected events take the identical path, so anything the scripted scenario can do, a demo facilitator or Bob can do live. Task generation is idempotent: the task set is discarded and rebuilt every cycle with deterministic IDs, infrastructure entity types are excluded, and task type selection requires evidence confidence of at least 0.5 before falling back to asset status. Reset restores all entity state from the scenario file, clears observations, decisions and the audit table, and marks every scripted event unprocessed.

TWO RESET CAVEATS FOUND IN CODE

Injected events persist as `EVT-INJ-*` rows and survive a reset, so they are replayed as if scheduled. And the in-memory audit log is not cleared on reset, so it accumulates across cycles. Neither breaks a demo run, but both should be tightened before the system is used for repeated measured comparisons.

5.7 API surface — 24 operations, complete

One FastAPI application serves both the dashboard and the MCP layer, so there is a single source of truth for state.

Method	Path	Purpose
GET	/health	Status, simulated clock, asset / observation / pending-event counts
GET	/situation	Top-10 priorities, blocked infrastructure, active conflicts, unavailable resources, plan summary
GET	/entities	All assets, roads and bridges
GET	/entities/{id}	Single entity with its priority and full evidence set
GET	/evidence/{id}	Fused evidence summaries plus the raw observations behind them
GET	/priorities	Full priority ranking with assigned ranks
GET	/resources	All resources with status, location and capabilities
GET	/routes	Route query by origin, destination and vehicle type
POST	/routes	Same via request body
POST	/simulate/next	Process the next scripted event and return the new situation
POST	/simulate/auto	Drain all remaining events
POST	/simulate/event	Inject a custom event
POST	/simulate/reset	Reset to initial scenario state
POST	/optimize	Run the optimiser and the baseline, return both with improvement percentage
GET	/decisions	Recent decisions with rationale
GET	/decisions/{id}	Single decision with a generated natural-language explanation
GET	/benchmark	Run the AI-vs-baseline benchmark on current state
GET	/timeline	Full event timeline with processed flags
GET	/map-data	GeoJSON feature collection for the dashboard map
POST	/ai/question	Natural-language operator Q&A against live state
GET	/ai/situation	Generated situation summary
GET	/audit	Recent audit entries
GET	/tasks	Current generated task set
GET	/plan	Current plan, baseline plan and comparison

5.8 IBM Bob MCP integration — 14 tools, complete

The MCP server is a JSON-RPC 2.0 stdio loop implementing `initialize`, `tools/list`, `tools/call` and the initialised notification, with correct error codes for unknown methods, unknown tools, tool exceptions and parse failures. Every tool resolves through a single HTTP helper against the backend, and every tool returns Markdown formatted from live response data. There are no hardcoded answers anywhere in the file. Connection failures are caught and reported as actionable messages rather than propagating as protocol errors.

Tool	Parameters	Backing endpoint
get_current_situation	—	GET /situation
get_priority_sites	top_n = 5	GET /priorities
get_entity_evidence	entity_id *	GET /evidence/{id}
get_route	origin *, destination *, vehicle_type	GET /routes
get_resource_status	—	GET /resources
generate_response_plan	—	POST /optimize
recalculate_allocation	—	POST /optimize
simulate_event	event_type *, entity_id, observation_type, value, confidence, severity, description, source_type	POST /simulate/event
simulate_next_event	—	POST /simulate/next
explain_decision	decision_id *	GET /decisions/{id}
get_benchmark_metrics	—	GET /benchmark
ask_operator_question	question *	POST /ai/question
get_timeline	—	GET /timeline
reset_simulation	—	POST /simulate/reset

* required. The 14 tools map to 13 distinct endpoints: `generate_response_plan` and `recalculate_allocation` both call `POST /optimize` and differ only in presentation framing. `top_n` is applied as a client-side slice rather than a query parameter.

WHY THE INTEGRATION IS LOAD-BEARING

Bob is not a veneer over static text. Asking “*why is HP1 the top priority?*” reaches the evidence endpoint and returns actual observation IDs, source types, fused confidence and conflict status. Asking “*can A1 reach it?*” executes a real Dijkstra query and names the specific bridges and roads that were excluded. Asking Bob to “*simulate road R7 blocking*” injects a real event and the whole pipeline replans. The natural-language layer is a window onto the deterministic engine, which is exactly the boundary Section 3.3 sets out.

5.9 React dashboard — complete

React 19 with Vite, TypeScript throughout, and eleven components. All of them render live backend data; there are no placeholder or mock components and no unfinished markers in the source.

Component	Renders
DisasterMap	Leaflet map with dark CARTO tiles; roads as polylines (blocked roads drawn dashed and heavier), assets / bridges / resources as circle markers sized by priority and coloured by status, with popups
PriorityPanel	Active-conflict banner plus the top 12 priority cards, each with a score bar and its priority / urgency / accessibility / confidence breakdown
EntityPanel	Selected entity with status, population, priority bar, generated explanation, factor chips, fused evidence summaries, raw observations and canned question shortcuts
ResourcesPanel	All resources grouped by status with location and capability detail, overlaid on the map
SimControls	Next event, auto-play, reset, plus six preset injections and a full custom-event form covering event type, entity, observation type, value and confidence
MetricsPanel	Six live tiles: AI coverage, baseline coverage, AI travel time, travel saving, conflicts detected, mean decision confidence
AIChatPanel	Operator chat transcript with pending state, wired to the Q&A endpoint
ErrorBoundary, LoadingScreen, BackendError	Render-error recovery, connection state and a retry path for backend unavailability

State is plain React hooks with no external store. A 5-second interval refreshes situation, resources, timeline and map data together; the benchmark is fetched on mount and after every mutation. The Vite dev server proxies `/api` to the backend, and the backend will serve the built frontend from `/` when a production build is present.

5.10 Persistence and seeding — complete

Async SQLAlchemy 2.0 over aiosqlite with eight tables: `assets`, `roads`, `bridges`, `resources`, `observations`, `decisions`, `sim_events` and `audit_log`. Structured sub-objects are stored as JSON text columns. Schema creation runs at startup; seeding is idempotent and exits early if assets already exist, so repeated launches are safe. Derived state — fused evidence, priorities, tasks, plans — is intentionally not persisted, because it is cheaper and safer to recompute it from the observation record than to keep it in sync.

5.11 AI provider — complete with fallback

An abstract provider with three operations: situation summary, decision explanation and operator Q&A. The watsonx implementation lazily imports the IBM SDK and calls `ibm/granite-3-8b-instruct` at temperature 0.3. A deterministic implementation produces the same three outputs from templates and keyword routing over live state. Selection requires both an API key and a project ID to be present; failure at client construction or generation falls back to the deterministic provider transparently. The system is therefore fully functional with no credentials, which is how it is expected to be demonstrated.

6 Algorithm Reference

Every constant below is the literal default in `app/config.py` or the engine that owns it. All are configurable in one place; none are scattered through the code as magic numbers.

6.1 Evidence weighting

```
freshness(obs) = exp(-λ[obs_type] × age_hours)
weight(obs)    = reliability[source_type] × obs.confidence × freshness(obs)

support[v]     = Σ weight(obs) for obs whose canonical value == v
best_value     = argmax support
confidence     = min(1, support[best]/Σweight × support[best]/|group[best]|)
conflict       = support[second]/Σweight ≥ 0.30
```

Source reliability

Hospital	0.95
Authority report	0.92
Drone	0.90
Field report	0.88
Satellite	0.85
School	0.85
Sensor	0.80
Emergency call	0.75
Simulation (ground truth)	1.00
Unrecognised source	0.70

Freshness decay λ (per hour)

Observation type	λ	Half-life
Flood level	3.0	14 min
Bridge status	2.0	21 min
Capacity	2.0	21 min
Road status	1.5	28 min
Structural	0.5	1.4 h
Population	0.1	6.9 h
Default	1.0	42 min

Fast-moving physical conditions expire quickly; population estimates persist. This is the mechanism by which a fresh drone report outweighs a stale field report from a nominally comparable source.

6.2 Scoring

```
criticality = 0.25·population_impact // min(1, population / 10 000)
            + 0.25·service_criticality // static per entity type
            + 0.20·time_sensitivity    // static per entity type
            + 0.15·isolation_risk      // from accessibility evidence
            + 0.15·damage_severity     // severity × confidence, max over evidence

priority    = 0.30·criticality
            + 0.25·urgency              // max over evidence values and asset status
            + 0.20·impact               // normalised affected population
            + 0.15·accessibility        // max(0.1, 1 - route_risk) from CC1; 0 if unreachable
            + 0.10·confidence           // mean evidence confidence × (1 - 0.15·conflicts)
```

Service criticality runs from hospital 1.00 and command centre 0.95 down through hydropower 0.85, health post 0.80, shelter 0.75, school 0.60 and village 0.55 to river 0.10. Time sensitivity is a parallel table topped by hospital 1.00 and health

post 0.90. Both weight sets sum to exactly 1.0, and a unit test asserts it.

6.3 Routing cost

```
// effective Dijkstra weight per directed edge
blocked          → ∞                (edge removed from the subgraph)
unknown          → travel_time × 1.4
high_risk        → travel_time × 1.6
partially_blocked → travel_time × 1.8
open             → travel_time × (1 + risk)

// bridge state overrides the road it carries
bridge blocked   → road blocked, risk = 1.0
road.confidence  → min(road.confidence, bridge.confidence)
vehicle admitted → capacity_order[bridge] ≥ capacity_order[vehicle]
                  light 1 < medium 2 < heavy 3
```

6.4 Assignment

```
eligible(r, t)  = r.status == AVAILABLE and t.type ∈ capability[r.type]
feasible(r, t)  = route(r.location → t.location, vehicle[r.type]).feasible
score(r, t)     = t.priority × (0.4 + 0.6 × max(0, 1 - travel/120))

greedy: take (r,t) in descending score, strict 1:1, skip taken
improve: swap i,j while both remain eligible and
         swapped_weighted_travel < current × 0.95    // up to 10 passes

baseline       = tasks in priority order → nearest feasible resource, no refinement
```

Resource type	Permitted tasks	Routing class
Rescue team	Rescue, evacuation, verification	Medium
Ambulance	Medical response	Heavy
Medical unit	Medical response, evacuation	Medium
Engineering team	Engineering, verification	Heavy
Supply vehicle	Supply delivery	Heavy

The routing class matters operationally: ambulances and heavy vehicles are refused medium-capacity bridges, so the same destination can be reachable for a rescue team and unreachable for an ambulance. That asymmetry is enforced in the graph, not approximated in the optimiser.

7 Scenario & Benchmark Harness

7.1 The Bhote Valley simulation — implemented and seeded

15 ASSETS	14 ROAD SEGMENTS	2 BRIDGES	8 RESPONSE RESOURCES	13 SCRIPTED EVENTS	180 MINUTES SIMULATED
--------------	------------------------	--------------	----------------------------	--------------------------	-----------------------------

The network forms two chains out of the command centre: one via the district hospital, a health post, a school, a shelter and a village to the hydropower station, and one via a supply base through four villages, a school and a health post to a second supply base. Bridge B1 sits on the hospital-to-health-post road at heavy capacity; bridge B2 sits on a village-to-village road at medium capacity. Because the topology is chain-like, a single blockage genuinely isolates everything downstream — which is the point, and is what makes the accessibility term in priority scoring do visible work.

Total population across the 15 assets is 9,930, of which the district hospital accounts for 3,500. Thirteen of the 15 assets receive priority scores; the two supply bases are excluded as infrastructure. All eight resources start available at one of four bases.

7.2 Event sequence

Thirteen events at exact 15-minute intervals from T+0 to T+180, drawn from four source classes with deliberately varied reliability.

T+	Event	Entity	Source	Conf.	Operational consequence
0	Normal state	—	Simulation	1.00	Establishes the pre-disaster baseline plan
15	Flood detection	V3	Satellite	0.85	First evidence of degradation; village priority rises
30	Bridge damage report	B1	Field report	0.82	B1 partially blocked; routing cost on its road inflates
45	Conflicting bridge report	B1	Field report	0.50	Contradicts T+30. Conflict flagged, confidence drops, verification recommended
60	Road blockage	R7	Field report	0.90	Edge removed from the graph; the entire second chain must reroute
75	Hospital demand surge	HP1	Hospital	0.95	Health post overloaded; urgency spikes on the highest-reliability source
90	Resource unavailable	RT2	Field report	1.00	A rescue team is withdrawn mid-plan; optimiser must reassign
105	Bridge confirmed blocked	B2	Satellite	0.88	Hard blockage; road forced blocked at risk 1.0
120	Village isolation confirmed	V3	Field report	0.90	Isolation risk fires; accessibility term collapses
135	Satellite observation	V4	Satellite	0.80	Second village flooded; competing demand for scarce units
150	Alternate route opened	R5	Field report	0.70	Recovery. Edge returns to the graph and the plan improves
165	School evacuation	S1	Field report	0.92	Evacuation task generated, requiring a compatible resource type
180	Satellite observation	V5	Satellite	0.75	Conditions clear at the last village; priority recedes

The sequence is designed so that each mechanism in the system is exercised at least once and is visible in the dashboard as a state change: fusion at T+15, conflict detection at T+45, graph mutation at T+60 and T+105, reallocation at T+90, isolation at T+120, and recovery at T+150.

7.3 Benchmark harness — implemented, 18 metrics

The harness builds both plans on the same world state and compares them. Nothing is hardcoded; every figure is computed from the state reached by however many events have been processed.

Metric group	Measures
Coverage	Tasks assigned, total tasks, coverage rate, critical sites covered, critical coverage rate, unmet critical demand
Efficiency	Total travel time, priority-weighted response time, resource utilisation
Safety	Blocked-route violations — allocations whose route is infeasible
Evidence quality	Conflict detection rate, total observations, total evidence items, mean decision confidence
Responsiveness	Replanning events completed
Improvement	Travel-time reduction percentage, coverage delta, conflicts detected, blocked-route violations avoided

INTEGRITY OF THE COMPARISON

We checked specifically for synthetic inflation of our own result, because that is the usual failure mode of self-reported benchmarks. The baseline is a real nearest-resource algorithm run on identical state, and no multiplier is applied to either side in the returned output. "Critical" is defined structurally as hospital, health post and shelter entities — four sites in this scenario — and not chosen to flatter the result.

One honest asymmetry to disclose when presenting: mean decision confidence falls back to 0.5 for the baseline, because the baseline algorithm does not construct decision explanations at all. That metric should be read as a property of the AI plan only, not as a comparison.

Benchmark figures are deliberately absent from this document. They depend on how far the simulation has advanced, and quoting a number here would invite exactly the kind of unverifiable claim the project is designed to avoid. Run `make benchmark` and report what it returns.

8 Verification Status

64 TOTAL TESTS	43 ENGINE UNIT TESTS	21 API INTEGRATION TESTS	5 / 5 ENGINES COVERED
-----------------------------	-----------------------------------	---------------------------------------	------------------------------------

8.1 What the unit tests prove

Forty-three tests exercise the engines as pure functions, with local factories rather than database fixtures. They cover the exact freshness formula including the slow-decay population case, the source-reliability constants and the weight product, fusion of single / empty / unanimous / conflicting / stale / equal-weight inputs, value canonicalisation, criticality ordering and bounds, priority bounds and explanation generation, routing across six cases including blocked-road exclusion, blocked-bridge propagation, alternate-route discovery and live update on bridge change, and seven optimiser cases including capability enforcement, blocked-route avoidance, no double assignment, exclusion of unavailable resources, and comparison against the baseline. One test asserts that both weight sets sum to 1.0, which catches the most likely form of silent configuration error.

8.2 What the integration tests prove

Twenty-one async tests drive the real application through an ASGI transport against a freshly seeded database. They cover every read endpoint, assert that priorities are returned in descending order, process a scripted event, inject a road blockage, inject a two-report bridge contradiction and assert the entity is flagged `conflicting`, run the optimiser, ask an operator question, reset the simulation, and finally run an end-to-end pass of reset, seven events, situation, route, optimise and Q&A. The benchmark endpoint is exercised with a skip guard for the no-task state.

VERIFICATION GAPS — STATED PLAINLY

The suite was not executed during preparation of this document. The environment available had no installable Python package manager, so the backend dependencies could not be provisioned and the 64-test claim is a count of defined test functions, verified by inspection, not a passing run. Anyone presenting this project should run `make test` and confirm.

Three known fragilities in the test setup, found by reading it: the integration tests share module-global world state and are therefore order-dependent; they assume the working directory is `src/backend` for the SQLite file; and they define a custom `event_loop` fixture, which is deprecated in `pytest-asyncio` 0.23 and removed in 1.x. Since `requirements.txt` pins only a lower bound, a fresh install today may pick up a version that breaks the integration suite. This is the single highest-value fix before a live demo.

Also unverified: CI runs no tests at all. The GitHub workflow validates file presence, YAML parsing, required submission fields and placeholder replacement — six structural checks with no build, lint, type-check or test step. Green CI on this repository does not indicate working code.

9 Gaps, Drift & Limitations

This section exists because a proposal that overstates its own completeness is worth less than one that does not. Everything here was found by comparing the repository's documentation against its code.

9.1 Documented but not implemented

Claim	Reality	Impact
<code>POST /actions/allocate</code> for operator confirm / reject	ABSENT	The most significant gap. The architecture describes confirm / reject cards feeding accepted allocations back into state, and the dashboard is built around advisory plans, but there is no write endpoint. The human-in-the-loop loop is open: recommendations can be read and reasoned about, not actioned in-system.
SSE stream pushing deltas to the dashboard	ABSENT	No SSE or WebSocket route exists. The dashboard polls four endpoints every 5 s, which is adequate at this scale but means up to 5 s of staleness and redundant transfer.
Durable audit log	PARTIAL	The <code>audit_log</code> table is created and cleared on reset but never written to. <code>GET /audit</code> serves the in-memory log only, so the post-incident review trail does not survive a restart.
Top-3 ranked routes with per-segment confidence	PARTIAL	Routing returns a single optimal path plus the list of roads excluded. Per-segment confidence is held on the edges but not returned per hop, and there is no k-shortest-paths implementation.
Tick loop at a configurable interval	DIFFERS	The engine is event-driven and jumps the clock to each event's offset. Behaviourally equivalent for the scripted scenario; the document is simply describing a design that was superseded.
Task duration estimates used in scheduling	UNUSED	Durations are computed and stored per task but never consumed. There is no scheduling, no time windows and no sequencing of multiple tasks per resource — assignment is strictly one-to-one.

9.2 Documentation drift

`demo/README.md` is the least reliable document in the repository and should be rewritten before submission. It refers to locations that do not exist in the scenario ("Balephi Junction", "Sindhupalchok North"), states seven villages where there are five, describes a six-hour timeline where the scenario ends at T+180, walks through a ranked route list and confidence bars that the UI does not have, scripts Bob queries for resource assignment and audit-log retrieval that no MCP tool supports, and describes the baseline as random allocation when it is nearest-resource. `demo/demo-video-link.txt` additionally repeats incorrect counts of 14 assets and 12 roads.

Correct figure	Verified	Incorrectly stated as
Assets	15	14 — in <code>README.md</code> and <code>demo/demo-video-link.txt</code>
Road segments	14	12 — in <code>README.md</code> and <code>demo/demo-video-link.txt</code>
Villages	5	7 — in <code>demo/README.md</code>
Simulated duration	180 min	6 hours — in <code>demo/README.md</code>
Baseline algorithm	Nearest-resource	Random allocation — in <code>demo/README.md</code>
REST operations	24	22 listed in <code>docs/architecture.md</code> ; all 22 exist, two are simply undocumented

9.3 Engineering and security notes

SECURITY POSTURE — DISCLOSE BEFORE ANY DEPLOYMENT

There is no authentication or authorisation on any endpoint. CORS is configured with a wildcard origin while also allowing credentials, and the state-mutating operations — `POST /simulate/event`, `/simulate/reset`, `/optimize` — are open to any caller that can reach the port. This is acceptable for a local single-operator demo and is not acceptable for the hosted deployment the demo materials anticipate. If the project is deployed publicly, this needs an auth layer and an origin allowlist first.

- **Unpinned dependencies.** All eleven backend packages use open `>=` ranges, so two installs a month apart can resolve to different versions. The `pytest-asyncio` case in Section 8.2 is a concrete instance of this already causing latent breakage.
- **Hardcoded environment paths.** The `Makefile` hardcodes `/opt/conda/bin/python3` and uses Unix-only commands; `mcp_config.json` hardcodes devcontainer absolute paths. Neither resolves on a Windows checkout, so both need editing before a local run outside the original container.
- **Undeclared MCP dependency.** `src/mcp/` has no requirements file but needs `httpx`, which it inherits from the backend environment. Running the MCP server in isolation will fail.
- **watsonx SDK not declared.** `ibm-watsonx-ai` is absent from `requirements.txt`, so the watsonx path requires a manual install. The deterministic fallback masks this, which is convenient but means the watsonx branch is unlikely to have been exercised.
- **Optimiser decision records go stale after a swap.** Local improvement rewrites the allocations but returns the decision list untouched, so a persisted `DecisionDB` row can describe the pre-swap pairing. Since explainability is the project's central claim, this is a correctness bug in the part that matters most.
- **Frontend loose ends.** `recharts` is installed but never imported, so the metrics panel is numeric tiles rather than charts. Timeline data is fetched every 5 s and never rendered. `VITE_API_BASE_URL` is read by the API client but is absent from `.env.example`, so a production build served outside the dev proxy has no working API base.
- **Benchmark writes to the working directory.** `benchmark_results.json` is written to the process CWD as a side effect of running the benchmark endpoint.

9.4 Scope limitations, by design

- **Closed simulation.** No live ingestion of sensor, satellite, telephony or weather feeds. Observations arrive only from the scenario script or manual injection.
- **Synthetic data throughout.** Every entity, coordinate, population and event is invented. The benchmark measures algorithmic behaviour on a constructed scenario, not real-world outcome improvement.
- **Greedy rather than exact optimisation.** Assignment is heuristic, not a solved integer program. At eight resources and a handful of tasks the gap is negligible, but the system cannot claim optimality.
- **Accessibility measured from a single origin.** The accessibility term in priority scoring routes from the command centre only, so it reflects reachability from headquarters rather than from whichever resource would actually respond.
- **Confidence is not calibrated.** As set out in Section 5.2, fused confidence is a support score. It should not be presented as a probability.
- **Map tiles require internet.** The dark CARTO basemap will not load offline; the rest of the dashboard degrades gracefully.
- **Demo artefacts incomplete.** Three screenshots exist. The demo video link and live demo URL are both placeholders, and CI's placeholder check does not catch either.

10 Roadmap

Ordered by value per unit of effort. The first group closes the loop the architecture already promises; the second makes the system defensible outside a demo; the third extends the research claim.

10.1 Close the human-in-the-loop loop

Work item	Effort	Why it matters
Allocation write-back endpoint <i>POST /actions/allocate + reject</i>	Small	Turns the dashboard's advisory cards into real decisions, makes resource state advance through deployment, and gives the replanning cycle something to react to. Everything needed already exists; only the endpoint and the state transition are missing.
Persist the audit log	Small	The table and the in-memory records both exist; they simply are not connected. Post-incident review is a stated goal and currently does not survive a restart.
Fix stale decision records after swap	Small	Explainability is the project's central claim, and a persisted explanation that describes a superseded pairing directly undermines it.
Rebuild <code>demo/README.md</code> from the code	Small	A judge who follows the current demo script will look for entities and features that do not exist. This is the highest-risk document in the repository.

10.2 Make it safe and reproducible

Work item	Effort	Why it matters
Pin dependencies; remove the deprecated <code>event_loop</code> fixture	Small	Guarantees the 64-test suite runs the same way on a fresh machine. Currently a clean install may resolve a pytest-asyncio version that breaks the integration tests.
Run tests, lint and build in CI	Small	CI today validates file presence only. Adding pytest, oxlint and <code>tsc -b</code> converts a green badge from decoration into evidence.
Authentication and an origin allowlist	Medium	Mandatory prerequisite for the hosted deployment the demo materials assume. Unauthenticated state mutation behind a wildcard CORS policy cannot be exposed publicly.
Portable Makefile and MCP config	Small	Removes the hardcoded conda and devcontainer paths that block a local run on a fresh checkout.
Isolate integration test state	Medium	Removes order dependence on shared global world state, so failures point at the defect rather than at test sequencing.
Record the demo video; deploy or remove the URL placeholder	Small	Both artefacts are currently placeholders that CI does not catch.

10.3 Extend the analytical claim

Work item	Effort	Why it matters
k-shortest routes with per-segment confidence	Medium	Delivers the confidence-versus-time tradeoff the solution overview describes and the coordinator actually wants: not one route, but the honest options.
Calibrate fused confidence	Medium	Make corroboration raise confidence, so the value can be defended as a probability rather than a support score. This is the change that would most strengthen the core research claim.
Per-resource accessibility	Medium	Score reachability from the responding unit rather than from the command centre, which is what the dispatch decision actually depends on.
Scheduling with task durations	Large	Durations are already computed. Consuming them permits multiple sequential tasks per resource and time-window feasibility, lifting the model from one-shot assignment to a response schedule.
Exact assignment solver	Medium	Min-cost-flow or ILP would let the system claim optimality and would quantify how much the greedy heuristic actually leaves on the table.
Live data adapters	Large	The observation model is already source-agnostic with per-source reliability, so real feeds are an adapter problem rather than a redesign. This is the path from simulation to pilot.
Server-sent events	Small	Replaces 5-second polling with push, removing staleness and redundant transfer on four endpoints.

RECOMMENDED NEXT FOUR ACTIONS

If only a day of work is available before submission: pin the dependencies and fix the `event_loop` fixture so the suite provably runs; add the allocation write-back endpoint so the human-in-the-loop story is demonstrable rather than described; rewrite `demo/README.md` against the actual scenario; and correct the asset and road counts in `README.md` . Together these convert three of the four weakest points in the submission into strengths, and none is large.

11 Appendix

11.1 Repository layout

```
bob-ai-hackathon-shutdown/
├─ src/
│  ├─ backend/
│  │  ├─ app/
│  │  │  ├─ main.py           FastAPI – 24 operations
│  │  │  ├─ config.py        every tunable parameter
│  │  │  ├─ models/domain.py  10 enums, 16 Pydantic v2 models
│  │  │  └─ engines/
│  │  │     ├─ evidence_fusion.py  weighting, decay, conflicts
│  │  │     ├─ priority_engine.py  criticality & priority
│  │  │     ├─ graph_engine.py    NetworkX accessibility
│  │  │     ├─ resource_optimizer.py greedy + swap, baseline
│  │  │     ├─ simulation.py      world state, events, tasks
│  │  │     └─ benchmark.py       18 metrics, AI vs baseline
│  │  └─ db/                   async SQLAlchemy, 8 tables, seeding
│  │     └─ utils/ai_provider.py  watsonx + deterministic fallback
│  └─ tests/                   43 unit + 21 integration
│     ├─ seed_runner.py
│     ├─ benchmark_runner.py
│     ├─ requirements.txt
│     └─ pytest.ini
├─ frontend/src/              App.tsx (11 components), api.ts, CSS
├─ mcp/                      server.py (14 tools), mcp_config.json
├─ data/scenario_nepal_inspired.json  15 assets, 14 roads, 13 events
├─ .env.example
├─ docs/                     problem statement, solution overview, architecture, setup
├─ demo/                     guide, 3 screenshots, video + URL placeholders
├─ presentation/             deck (pptx + pdf)
├─ Makefile                  11 targets
└─ submission.yaml
```

11.2 Running the system

Command	Action
make install	Verifies the scenario file exists, installs backend requirements, installs frontend packages
make seed	Deletes the SQLite database and reseeds from the scenario file
make dev-backend	uvicorn with reload on port 8000; interactive API docs at <code>/docs</code>
make dev-frontend	Vite dev server on port 5173, proxying <code>/api</code> to the backend
make test	Seeds, then runs all 64 tests
make benchmark	Seeds, processes up to 10 events, prints the benchmark JSON
make build-frontend	Type-checks and builds; the backend then serves the bundle from <code>/</code>
make clean	Removes the database, benchmark output, frontend build and Python caches

The MCP server is launched by the Bob client using `src/mcp/mcp_config.json`, with `BACKEND_URL` pointing at the running backend. Both the interpreter path and the script path in that file must be corrected for the local machine, as

noted in Section 9.3. No credentials are required for any part of the system; watsonx is optional and the deterministic provider is used automatically in its absence.

11.3 Dependencies

Backend — 11 packages

fastapi, uvicorn[standard], pydantic, pydantic-settings, networkx, httpx, sqlalchemy, aiosqlite, python-dotenv, pytest, pytest-asyncio

All specified with open lower bounds. `ibm-watsonx-ai` is not declared and must be installed manually to exercise the watsonx path. There is no MCP SDK; the server implements JSON-RPC 2.0 directly.

Frontend

react 19, react-dom 19, leaflet, react-leaflet 5, recharts (*unused*); dev: vite, typescript, @vitejs/plugin-react, oxlint, type definitions

No router, no state library, no CSS framework and no test framework. Styling is hand-written CSS with custom properties.

11.4 Summary judgement

WHERE THE PROJECT STANDS

The analytical core is the strongest part of this submission and it is finished. Five engines implement a coherent, defensible thesis — that uncertainty belongs in the decision, not in a footnote — and they do it deterministically, with every weight traceable and every number reproducible. The MCP integration is genuinely load-bearing rather than decorative, and the dashboard makes the replanning visible rather than merely claimed.

The weaknesses are at the edges, not the centre: an unclosed human-in-the-loop write path, an audit trail that does not persist, a demo guide that describes a different system, and unpinned dependencies that put the test suite's reproducibility at risk. All four are small and well understood. None requires revisiting the design.

Prepared 18 September 2026 by reading the source tree at `bob-ai-hackathon-shutdown`. Counts and constants verified directly in code; the test suite was counted by inspection but not executed. No source files were modified. All scenario data referenced in this document is synthetic.